PROJECT REPORT

Ticket Counter System Using Queue (ARRAY)

SUBMITTED BY-

1.Priyanka Atri

2. Japneet Singh

UID:

1. 25BCA10008

2. 25BCA10364

SUBMITTED TO- Konika Rani

SECTION: 25 BCA 1(A)

DEPARTMENT: UIC

SEMESTER: 2

SUBJECT: Data Structure Lab

SUBJECT CODE- 25CAP-152

TABLE OF CONTENT

S.no.	Title	Pg.No .
1.	Acknowledgment	3
2.	Abstract	4
3.	Introduction	5
4.	Objectives	6-7
5.	Algorithm	8
6.	Time and Space Complexity	9-10
7.	Advantages	11
8.	Limitations	12
9.	Applications	13
10.	Conclusion	14
11.	Bibliography	15

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our teacher, Ms. Konika Rani, for providing valuable guidance, support, and encouragement throughout the completion of this project. Their clear explanations and continuous motivation helped us understand the concept of queue data structure and its practical implementation.

We are also thankful to our institution for providing the necessary resources and environment to successfully complete this project.

We would like to extend our thanks to our classmates and friends for their support and helpful discussions during the development of this project.

Lastly, we are grateful to our families for their constant encouragement and support, which helped us complete this project successfully.

Priyanka Atri

Japneet Singh

UID: 25BCA10008, 25BCA10364

First Year BCA

Chandigarh University

ABSTRACT

This project presents the implementation of a Ticket Counter System using the queue data structure in the C programming language. A queue is a linear data structure that follows the FIFO (First In First Out) principle, where the first customer to enter the queue is the first to be served.

In this system, each customer is assigned a unique ticket number, which is stored in a queue. The program performs basic queue operations such as enqueue (adding a ticket), dequeue (serving a customer), and display (showing the current queue status). These operations help simulate a real-life ticket counter scenario in an efficient and organized manner.

The queue is implemented using an array with fixed size, and two pointers—front and rear—are used to manage the queue. The program also handles special conditions such as overflow (when the queue is full) and underflow (when the queue is empty).

This project helps in understanding the practical application of queue data structures in real-world systems like ticket booking, banks, and call centers. It also improves programming skills and logical thinking by demonstrating how data structures are used to solve real-life problems efficiently.

INTRODUCTION

A queue is one of the most important linear data structure. It follows the principle of FIFO (First In First Out), which means that the element inserted first is the one that gets removed first. This behaviour is similar to real-life situations, such as people standing in a line at a ticket counter, where the person who arrives first is served first.

In computer programming, queues are widely used to manage data in an ordered manner. They are especially useful in situations where tasks must be processed sequentially, such as CPU scheduling, printer task management, call centre systems, and ticket booking systems.

This project focuses on implementing a Ticket Counter System using a queue in the C programming language. In this system, each customer is assigned a unique ticket number when they arrive. These ticket numbers are stored in a queue, ensuring that customers are served in the exact order in which they entered the queue. The system allows basic operations such as adding a customer (enqueue), serving a customer (dequeue), and displaying the current queue status.

The queue in this program is implemented using an array, which provides a simple and efficient way to store elements. Two variables, called front and rear, are used to keep track of the beginning and end of the queue, respectively. While this approach is easy to implement, it also has some limitations, such as fixed size and memory wastage, which can be improved using advanced techniques like circular queues.

Overall, this project helps in understanding how data structures like queues are applied in real-world scenarios and how they help in managing and organizing data efficiently.

OBJECTIVES

The main objectives of this project are as follows:

1. **To understand the concept of queue data structure**
   The primary objective is to learn how a queue works based on the FIFO (First In First Out) principle. It helps in understanding how data is organized and processed sequentially.

2. **To implement queue using arrays in C programming**
   This project aims to develop programming skills by implementing a queue using arrays. It includes handling operations like insertion (enqueue) and deletion (dequeue) using C language.

3. **To simulate a real-world ticket counter system**
   The project represents a real-life scenario where customers stand in a queue to get service. It helps in relating theoretical concepts with practical applications.

4. **To perform basic queue operations**
   The program demonstrates important queue operations such as:
   - **Enqueue (adding a customer/ticket)**
   - **Dequeue (serving a customer)**
   - **Display (showing current queue status)**

5. **To understand overflow and underflow conditions**
   The objective is also to learn how to handle situations when:
   - **The queue is full (overflow)**
   - **The queue is empty (underflow)**

6. **To improve problem-solving and logical thinking skills**
   By designing and implementing the system, students develop logical thinking and coding skills.

ALGORITHM

1. Enqueue (Take Ticket)

1. **Start**
2. **If rear == MAX - 1, print Queue is full**
3. **Else**
   - **If front == -1, set front = 0**
   - **Increase rear by 1**
   - **Add ticket at queue[rear]**
4. **Stop**

2. Dequeue (Serve Customer)

1. **Start**
2. **If front == -1 OR front > rear, print Queue is empty**
3. **Else**
   - **Print queue[front]**
   - **Increase front by 1**
   - **If front > rear, set front = rear = -1**
4. **Stop**

3. Display Queue

1. **Start**
2. **If queue is empty, print Queue is empty**
3. **Else**
   - **Print elements from front to rear**
4. **Stop.**

---

4. Main Algorithm

1. **Start**
2. **Show menu (Take Ticket, Serve, Display, Exit)**
3. **Take user choice**

4. **Call respective function**
5. **Repeat until Exit**

TIME AND SPACE COMPLEXITY

Enqueue Operation (Take Ticket)

- **You check if the queue is full**
- **Increase rear by 1**
- **Insert the ticket number**

Only a few fixed steps are performed, no loops are used.

✅ **Time Complexity = O(1)**
➡ **Because time does not depend on number of customers**

Dequeue Operation (Serve Customer)

- **You check if queue is empty**
- **Print the front ticket**
- **Increase front by 1**

👉 **Again, only constant steps are involved.**

✅ **Time Complexity = O(1)**

Display Operation

In the program:

- **A loop runs from front to rear**
- **Prints all elements in queue**

👉 **If there are n customers, loop runs n times**

✅ **Time Complexity = O(n)**

👉 Overall Time Complexity in The Project

Operation	Time Complexity
Take Ticket	O(1)
Serve Customer	O(1)
Display Queue	O(n)

2. Space Complexity

Memory Used in the Program

- **Queue is stored in:**
  int queue[MAX] where MAX = 5

👉 **This means memory is fixed for 5 elements**

- **Other variables used:**
  front, rear, ticket, choice → constant memory

Total Space Used

- **Array → O(n) (depends on size of queue)**
- **Variables → O(1)**

✅ **Final Space Complexity = O(n)**

ADVANTAGES

1. **Maintains Proper Order (FIFO) :-**
   The queue follows the First In First Out principle, so customers are served in the exact order they arrive. This ensures fairness in the system.

2. **Easy to Implement :-**
   The queue is implemented using an array and simple logic (front and rear pointers), making it easy to understand and code.

3. **Reduces Confusion :-**
   Since customers are served in order, there is no conflict or confusion about who will be served next.

4. **Better Management of Customers :-**
   The system helps in organizing and managing multiple customers efficiently.

5. **Simple Memory Usage :-**
   Uses a fixed-size array, which makes memory allocation simple and predictable.

LIMITATIONS

- **Fixed Size (Array Limitation) :-**

The queue has a fixed size (MAX = 5), so it cannot store more elements once full.

- **Memory Wastage :-**

After deletion, empty spaces at the beginning of the array cannot be reused in a linear queue.

- **Overflow Condition :-**

When the queue is full, no new customers can be added even if there is unused space earlier.

- **Underflow Condition :-**

When the queue is empty, deletion cannot be performed, which may cause errors if not handled properly.

- **Not Flexible :-**

The size of the queue cannot be increased dynamically, making it less suitable for large systems.

APPLICATIONS

- **Ticket Booking Systems :-**

Used in railway, bus, or movie ticket counters to manage customers in order.

- **Bank Queue Management :-**

Helps in serving customers in the order they arrive.

- **CPU Scheduling :-**

Operating systems use queues to manage processes.

- **Printer Queue :-**

Print jobs are stored and executed in order.

- **Call Centers :-**

Incoming calls are handled in the order they are received.

CONCLUSION

The Ticket Counter System using queue is a simple and effective way to manage customers in a structured manner. It follows the FIFO principle, ensuring fairness and proper order of service. The program is efficient and easy to implement, but it has some limitations such as fixed size and memory wastage. These limitations can be overcome by using advanced data structures like circular queue. Overall, this project helps in understanding the practical use of queues in real-world applications.

BIBLIOGRAPHY

1. https://chatgpt.com/
2. https://www.scribd.com
3. https://en.wikipedia.org